ITSM

UNIDAD2:TAREA 2.2

ESTRUCTURA DE DATOS

RUBEN MIGUEL RIOJAS RODRIGUEZ

PRESENTADO POR

-XIMENA SANCHEZ LUGO

INTRODUCCIÓN

El modelo Entidad-Relación Extendido (E-RE) es la base del diseño de la base de datos relacional; sin embargo, en el desarrollo orientado a objetos (POO) la estructura del sistema se representa con **diagramas de clases UML**. Este reporte parte del diagrama E-RE de la clínica veterinaria (Tarea 2.1) y lo convierte a un diagrama de clases, estableciendo la correspondencia entre entidades y clases, atributos y propiedades, y relaciones y asociaciones (composición, agregación y herencia). Después analiza qué conserva, qué gana y qué pierde cada notación, y cómo se aprovecha la correspondencia al construir el software.

DIAGRAMAS: ORIGEN (E-RE) Y DESTINO (UML)

Las páginas siguientes muestran el diagrama E-RE de origen, el diagrama de clases UML completo y dos detalles ampliados del UML. Ambos se elaboraron en draw.io y sus archivos .drawio se entregan en GitHub.

Figura 1. Diagrama E-R Extendido (origen: Tarea 2.1)

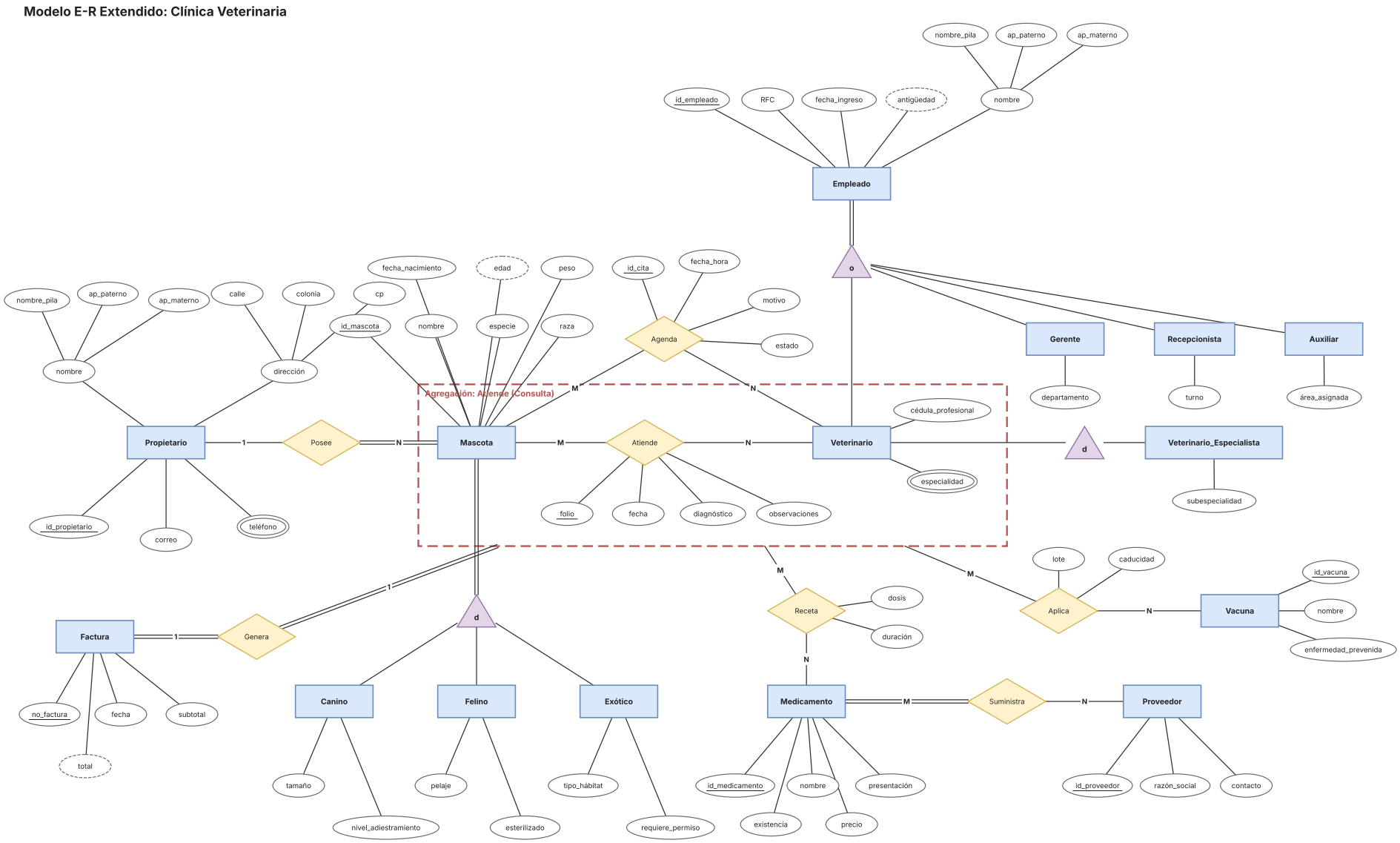


Figura 2. Diagrama de clases UML (destino)

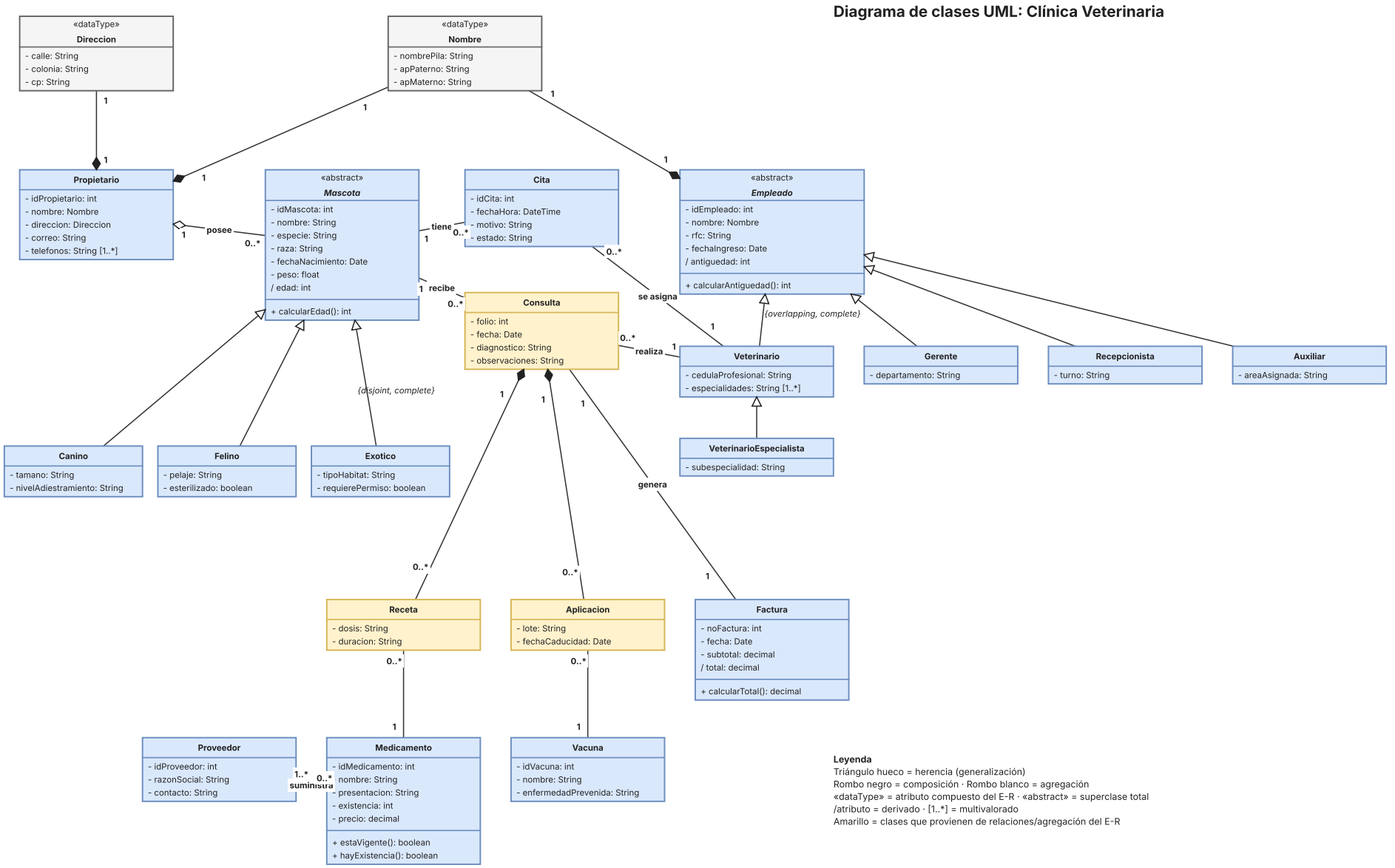


Figura 3. Detalle UML izquierdo: Propietario, Mascota, Cita, Consulta, herencia disjunta

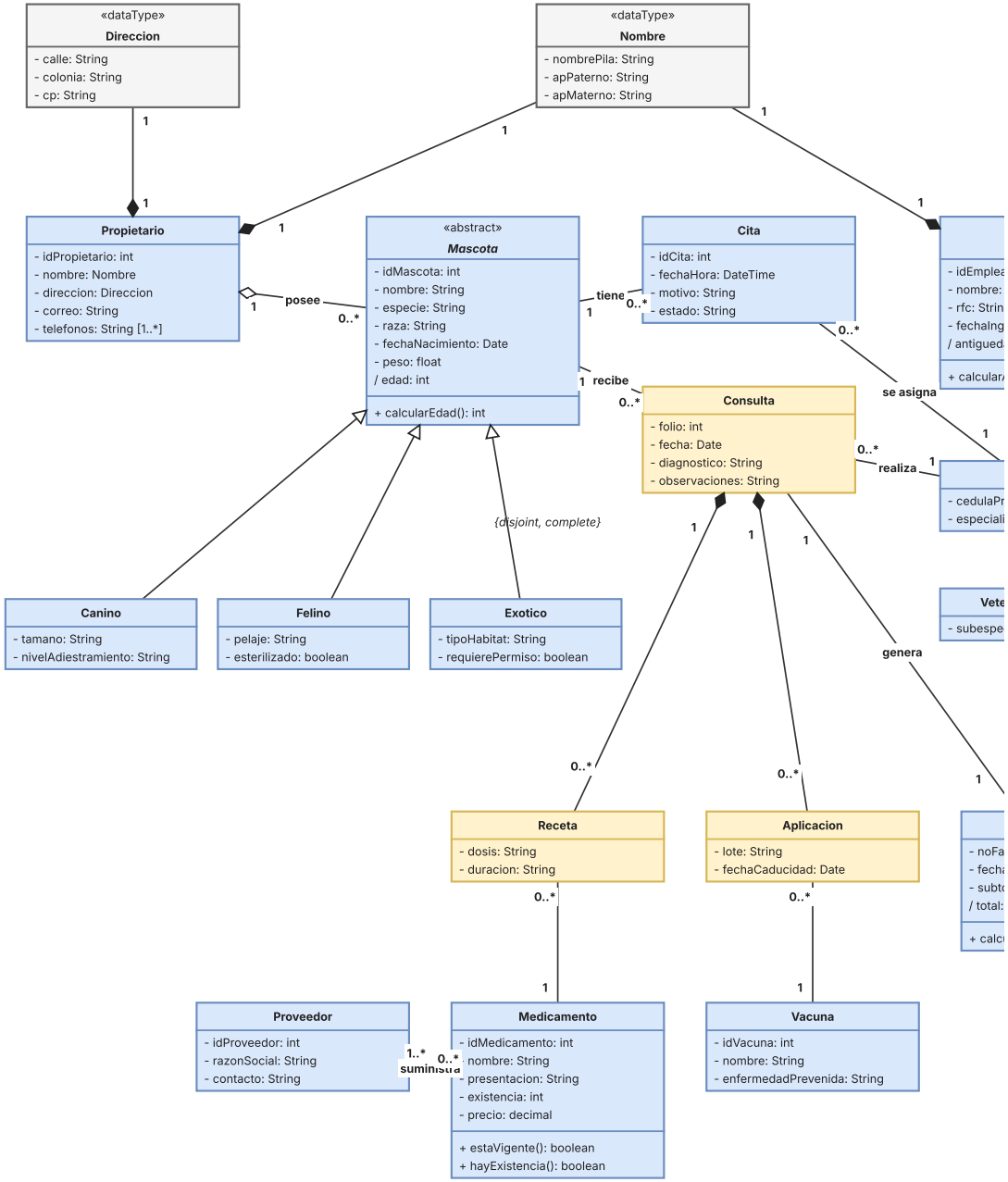


Figura 4. Detalle UML derecho: Empleado, jerarquia solapada, Receta, Aplicacion, Factura

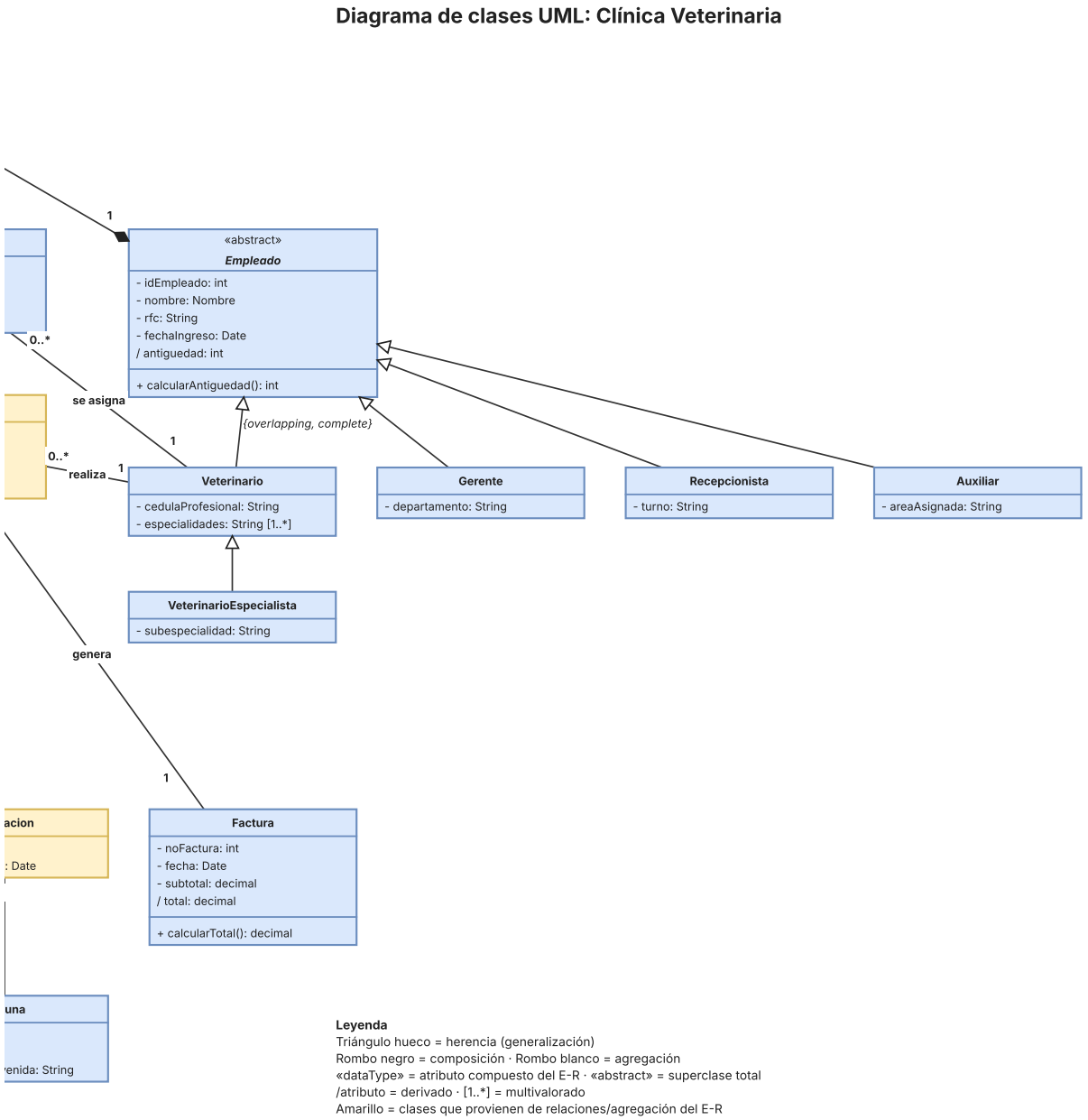


TABLA COMPARATIVA DE CORRESPONDENCIA DIRECTA

a) Elementos estructurales.

Elemento E-R Extendido	Notación UML (clases)	Aplicación en la clínica veterinaria
Entidad (rectángulo)	Clase	Propietario, Medicamento, Proveedor, Vacuna, Factura
Superentidad con especialización total	Clase abstracta (nombre en cursiva, «abstract»)	Mascota y Empleado
Atributo simple	Atributo con visibilidad y tipo	- peso: float, - correo: String
Atributo clave (subrayado)	Atributo identificador (UML no tiene símbolo de PK)	- idPropietario: int, - idMascota: int
Atributo compuesto (elipse con subatributos)	Clase «dataType» unida por composición	Nombre (nombrePila, apPaterno, apMaterno) y Direccion (calle, colonia, cp)
Atributo multivalorado (doble elipse)	Atributo con multiplicidad [1..*]	telefonos: String [1..*]; especialidades: String [1..*]
Atributo derivado (elipse punteada)	Atributo con prefijo / y método de cálculo	/ edad con calcularEdad(); / antiguedad; / total con calcularTotal()
Relación M:N con atributos	Clase de asociación o clase intermedia (reificación)	Agenda → Cita; Atiende → Consulta; Receta; Aplica → Aplicacion

b) Relaciones, restricciones y jerarquías.

Elemento E-R Extendido	Notación UML (clases)	Aplicación en la clínica veterinaria
Relación 1:N (Posee)	Agregación con multiplicidad 1 y 0..*	Propietario 1 ◇— 0..* Mascota (rombo blanco)
Relación 1:1 (Genera)	Asociación con multiplicidad 1 a 1	Consulta 1 — 1 Factura
Relación M:N (Suministra)	Asociación 0..* a 1..*	Medicamento 0..* — 1..* Proveedor
Participación total	Multiplicidad mínima 1 (o 1..*)	Mascota siempre tiene propietario; Medicamento 1..* Proveedor
Participación parcial	Multiplicidad mínima 0	Propietario 0..* Mascota; Consulta 0..* Receta
Especialización disjunta y total (d)	Herencia con restricción {disjoint, complete}	Canino, Felino y Exotico heredan de Mascota
Especialización solapada (o)	Herencia con restricción {overlapping, complete}	Veterinario, Gerente, Recepcionista y Auxiliar heredan de Empleado
Jerarquía de varios niveles	Cadena de herencia	Empleado → Veterinario → VeterinarioEspecialista
Agregación E-R (relación como entidad abstracta)	Clase propia con composición hacia sus partes (rombo negro)	Consulta ◆ Receta; Consulta ◆ Aplicacion; Consulta — Factura
Atributo compuesto (parte del todo)	Composición (rombo negro)	Propietario ◆ Direccion; Propietario ◆ Nombre; Empleado ◆ Nombre

CORRESPONDENCIA ENTIDAD → CLASE

Objeto en el E-RE	Clase UML	Observación
Propietario	Propietario	Contiene Nombre y Direccion por composición
Mascota	«abstract» Mascota	Superclase; hereda a Canino, Felino, Exotico
Canino, Felino, Exótico	Canino, Felino, Exotico	Atributos propios de cada tipo
Empleado	«abstract» Empleado	Superclase con jerarquía solapada
Veterinario, Gerente, Recepcionista, Auxiliar	Veterinario, Gerente, Recepcionista, Auxiliar	Un empleado puede tener varios roles; ver análisis
Veterinario_Especialista	VeterinarioEspecialista	Hereda de Veterinario (nivel 3)
Relación Agenda	Clase Cita	Asociada 0..* con Mascota y con Veterinario
Relación Atiende (agregación)	Clase Consulta	Dueña de Receta y Aplicacion (composición); genera Factura
Relación Receta	Clase Receta	Atributos dosis y duracion; apunta a un Medicamento
Relación Aplica	Clase Aplicacion	Atributos lote y fechaCaducidad; apunta a una Vacuna
Medicamento, Proveedor, Vacuna, Factura	Medicamento, Proveedor, Vacuna, Factura	Con métodos estaVigente(), hayExistencia(), calcularTotal()
Atributos compuestos nombre y dirección	«dataType» Nombre y Direccion	Tipos de valor reutilizables

ANÁLISIS DE COINCIDENCIAS Y DIFERENCIAS LÓGICAS

Coincidencias

- **Estructura equivalente:** las entidades, sus atributos y sus relaciones se corresponden uno a uno con clases, propiedades y asociaciones; el modelo conceptual no se rehace, se traduce.
- **Restricciones preservadas:** las cardinalidades del E-R se vuelven multiplicidades (1:N → 1 y 0..*), y la participación total o parcial se conserva en la multiplicidad mínima.
- **Generalización/especialización:** equivalen directamente a la herencia; las restricciones d/o y total/parcial pasan a {disjoint}/{overlapping} y {complete}/{incomplete}.
- **Atributos especiales:** los compuestos, multivalorados y derivados tienen representación en ambas notaciones.

Diferencias

- **Comportamiento:** el E-R solo describe datos. UML añade operaciones como + calcularTotal() en Factura y + hayExistencia() en Medicamento.
- **Visibilidad y tipos:** UML indica si un atributo es privado o público y su tipo (int, String, Date); el E-R no.
- **Claves:** el E-R subraya la clave primaria; UML no tiene ese concepto porque los objetos se identifican por su referencia. En el diagrama se conservan los id como atributos para facilitar la persistencia.
- **Relaciones M:N con atributos:** en el E-R son un rombo con atributos; en UML se resuelven como clase de asociación o clase intermedia (aquí se usó la segunda, que es la que se implementa en código).
- **Agregación:** son conceptos distintos. En E-R, la agregación convierte una relación en una entidad abstracta para relacionarla con otras; en UML es una relación de parte-todo débil. Por eso

la agregación Atiende del E-R se materializa como la clase *Consulta* con composición hacia Receta y Aplicacion, y el rombo blanco de UML se usó aparte para Propietario ◇ Mascota.

- **Rol solapado:** la herencia de UML asigna una sola clase a cada objeto. Para que un veterinario sea también gerente, en el código se resuelve con roles (composición de roles) en vez de herencia múltiple; esto es una limitación a tener en cuenta.

IMPLICACIONES PARA EL DESARROLLO ORIENTADO A OBJETOS

La correspondencia permite derivar tanto el esquema relacional como las clases del dominio desde un mismo modelo. Al usar un ORM (Object-Relational Mapping), el mapeo típico es:

Construcción UML	Implementación en código	Implementación en la base de datos
Clase	Clase de dominio / entidad del ORM	Tabla
Herencia (Mascota → Canino...)	extends / herencia de clases	Tabla por jerarquía, o tabla por subclase con clave compartida
Composición con «dataType»	Objeto incrustado (@Embedded)	Columnas dentro de la tabla del propietario
Atributo multivalorado [1..*]	Colección (List<String>)	Tabla aparte con clave foránea
Atributo derivado /	Método o propiedad calculada, no persistente	No se almacena
Asociación 1 — 0..*	Referencia y colección	Clave foránea en el lado «muchos»
Asociación 0..* — 1..* (Suministra)	Colecciones en ambos lados	Tabla intermedia con dos claves foráneas
Composición Consulta ◆ Receta	Colección propia de la consulta (cascade)	Clave foránea; se elimina junto con la consulta

CONCLUSIÓN

La transición del E-RE a UML muestra que ambos modelos son complementarios: el E-RE garantiza una estructura de datos íntegra y la notación UML añade comportamiento, encapsulamiento y herencia, que son las bases del software orientado a objetos. La correspondencia entre entidades y clases, entre relaciones y asociaciones, y entre especialización y herencia permite mantener un solo diseño coherente de la base de datos y del código, y facilita el uso de ORMs. Las diferencias señaladas (claves, agregación, roles solapados) deben atenderse al implementar.

ENLACES DE ENTREGA

Entregable	Enlace
Repositorio GitHub (diagramas editables .drawio y PDF)	[PEGAR ENLACE DEL REPOSITORIO]
Video de YouTube (correspondencia BD ↔ clases, con cámara web visible en la esquina inferior derecha)	[PEGAR ENLACE DEL VIDEO]

REFERENCIAS BIBLIOGRÁFICAS

- Ambler, S. W. (2004). *The Elements of UML 2.0 Style*. Cambridge University Press.
- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.